



Research Description of the Accelera System Modules

Mohamed Ashraf Nesma Osama Mazen Adel Mohamed Khater
ID: 9220658 ID: 9220912 ID: 9220625 ID: 9220713

Supervisor: Dr. Salma Abdel monem

April 17, 2026

Abstract

Accelera is a hybrid Python/C++ machine learning framework that combines a high-level Python user interface with native execution components. The system is organized around five modules: Accelera Pipe, the Code Parallelizer, AutoML, Deployment, and the Benchmark Platform. Accelera Pipe expresses machine learning workflows as directed graph nodes for preprocessing, modelling, prediction, metric evaluation, branching, and merging. The Code Parallelizer analyzes C/C++ and supported Python code, extracts loop features, predicts parallelization opportunities, and emits OpenMP-annotated C++ code. This thesis documents the architecture, methodology, optimizations, usage patterns, and experimental results of the system modules, while reserving space for modules whose documentation is still under development.

Contents

1	Introduction	3
2	Literature Review	3
3	Methodology	3
3.1	Accelera Pipe Module	3
3.1.1	Module Responsibility	3
3.1.2	Graph Execution Model	3
3.1.3	Branching and Node Sharing	4
3.1.4	Memory-Lifetime Optimizations	5
3.1.5	Saving and Loading Pipelines	6
3.1.6	How to Use Accelera Pipe	6
3.2	Code Parallelizer Module	7
3.2.1	Module Responsibility	7
3.2.2	Loop Analysis and Pragma Selection	7
3.2.3	Classifier Model and Rule-Based Decision	7
3.2.4	Python-to-C++ Converter Support	8
3.2.5	Integration with Accelera Pipe	8
3.2.6	How to Use the Parallelizer	9

3.3	AutoML Module	9
3.4	Deployment Module	9
3.5	Benchmark Platform Module	9
4	Results and Experiments	9
4.1	Accelera Pipe Results	9
4.2	Code Parallelizer Results	11
4.3	AutoML Results	12
4.4	Deployment Results	12
4.5	Benchmark Platform Results	12
5	Conclusion	12

1 Introduction

Accelera was designed for workflows where developers want to keep the expressiveness of Python while delegating expensive execution decisions to compiled infrastructure. The project has two central execution ideas. First, machine learning workflows can be represented as graphs rather than as a single sequential script. This allows several preprocessing and model alternatives to be evaluated inside one pipeline run. Second, loop-heavy custom code can be inspected and transformed into OpenMP-enabled C++ when the code falls inside the supported conversion and parallelization subset.

The complete project is divided into five modules. Accelera Pipe is the graph-based pipeline module. The Code Parallelizer is the source-code analysis and OpenMP generation module. AutoML is intended to automate model and preprocessing decisions. Deployment covers packaging and serving of trained artifacts. The Benchmark Platform is intended to provide repeatable performance and correctness measurements across modules. The following sections organize the thesis using this module structure so that implementation details, algorithms, and experimental results can be expanded independently.

2 Literature Review

3 Methodology

3.1 Accelera Pipe Module

3.1.1 Module Responsibility

Accelera Pipe is implemented under `accelera/src/accelera_pipe/`. Its responsibility is to provide a Python builder API over a C++ graph backend. The user writes a pipeline in Python, but the graph structure, node scheduling, branch execution, and intermediate memory lifetime are handled by the native backend.

The core files are:

- `core/pipeline_base.py`: owns or receives a native `graph.Graph` instance and provides shared save/load behavior.
- `core/pipeline.py`: implements the public builder API: `preprocess`, `model`, `predict`, `metric`, `merge`, and `branch`.
- `core/executed_graph.py`: wraps a trained graph for repeated inference.
- `core/metric.py`: defines the metric abstraction.
- `wrappers/`: contains supervised and unsupervised metric adapters.

3.1.2 Graph Execution Model

Each pipeline operation becomes a graph node. A preprocessing node transforms the input matrix, a model node fits or applies an estimator, a prediction node performs inference, a metric node evaluates the prediction, and a merge node combines multiple branch outputs. The graph is built incrementally, so the Python API remains fluent while the backend receives enough structure to execute independent branches in parallel.

Table 1: Accelera Pipe builder methods

Method	Node type	Main role
<code>preprocess(name, func)</code>	PREPROCESS	Transform input data
<code>model(name, model)</code>	MODEL	Fit or apply estimator
<code>predict(name, test_data)</code>	PREDICT	Run model prediction
<code>metric(name, metric)</code>	METRIC	Score predictions
<code>merge(name, method)</code>	MERGE	Combine branch outputs
<code>branch(name, ...)</code>	Multiple	Create alternative graph paths

The pipeline returns two objects after training: the metric or prediction results and an `ExecutedGraph`. The executed graph contains the fitted objects required for inference. This separation is important because training uses temporary arrays, validation targets, and candidate branches, while inference should keep only the selected executable state.

Algorithm 1: Pipeline training and executed graph construction

-
- 1: Input: training data X, target y, graph G, selection strategy S
 - 2: Output: results R, executed graph E
 - 3:
 - 4: 1: Attach X and y to the input node of G
 - 5: 2: Execute ready preprocessing, model, prediction, metric, and merge nodes
 - 6: 3: For each candidate path:
 - 7: 4: collect validation metric and path metadata
 - 8: 5: Select path P according to strategy S
 - 9: 6: Clone the selected fitted nodes from P into a compact executed graph E
 - 10: 7: Remove temporary training arrays and non-required execution data from G
 - 11: 8: Return results R and executed graph E
-

3.1.3 Branching and Node Sharing

Branching is one of the main reasons Accelera Pipe uses a graph rather than a linear pipeline object. A user may compare several preprocessors, several models, or complete candidate sub-pipelines. The backend can then schedule independent branches concurrently. A key optimization added to the branch construction step is duplicate first-node sharing. If a branch starts with the same single preprocessing object as another branch, the graph creates the node once and connects the later branch consumers to the same output. This avoids running identical first-stage transformations multiple times.

The sharing rule is intentionally conservative. It is applied only to single branch nodes, the first node of a branch list, or a list containing one item. Later duplicated nodes inside two different lists are not merged automatically, because by that point their inputs may already be different. For example, `[p1, p2, p3]` and `[p4, p2, p3]` do not share `p2`; the first path feeds `p2` with `p1(X)`, while the second feeds it with `p4(X)`.

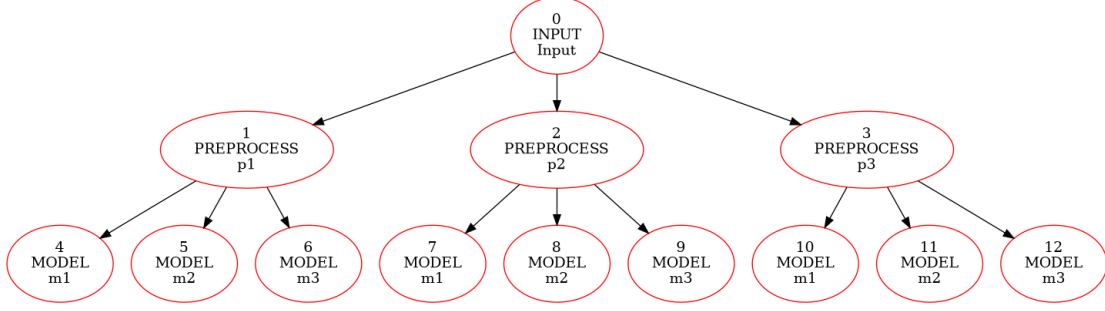


Figure 1: Branch graph before duplicate first-node sharing

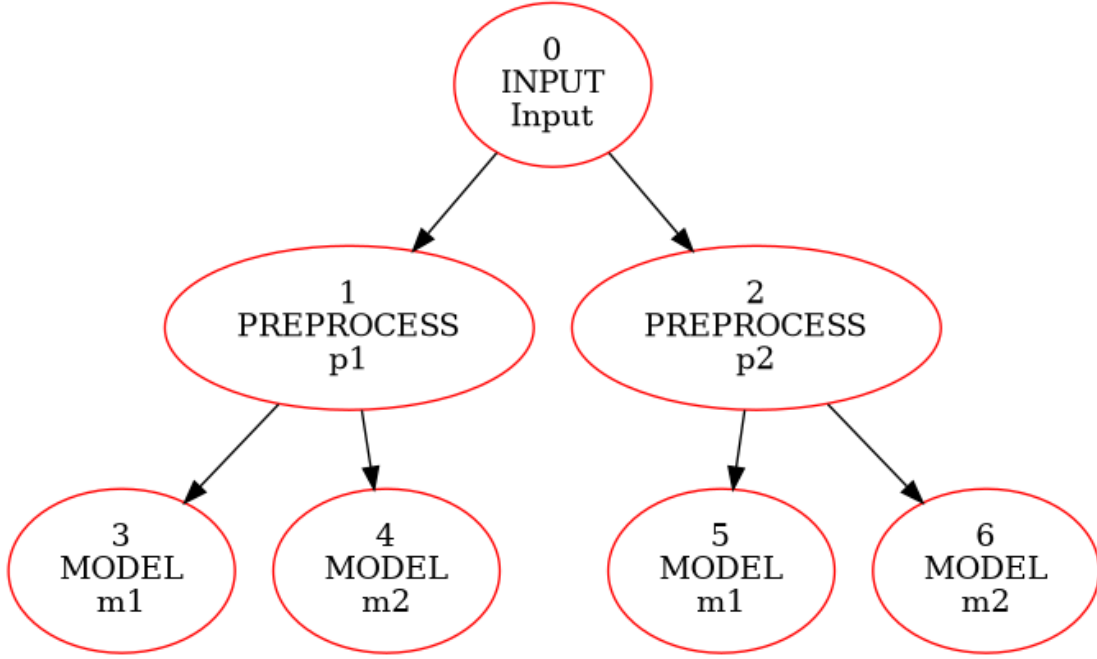


Figure 2: Branch graph after duplicate first-node sharing

3.1.4 Memory-Lifetime Optimizations

The largest memory pressure in Accelera Pipe appears during branch execution. Each preprocessing node can produce a transformed training array, and that array may be consumed by several downstream model nodes. The backend therefore tracks how many consumers still need each intermediate output. When the last consumer finishes, the output becomes dead and can be released.

Algorithm 2: Consumer-count based intermediate release

-
- 1: Input: executed node N, graph consumer count table C
 - 2: Output: graph with dead temporary arrays cleared
 - 3:
 - 4: 1: for each source node S that feeds N do
 - 5: 2: $C[S] = C[S] - 1$
 - 6: 3: if $C[S] == 0$ and S is releasable then
 - 7: 4: clear temporary data stored by S
 - 8: 5: end if
 - 9: 6: end for
 - 10: 7: if N is a model node and fit has completed then

```
11: 8:    clear the training input stored only for fitting N
12: 9: end if
```

Several smaller optimizations support this lifetime policy. Cache execution is optional and disabled by default using `cache=False`, because retaining node data can increase memory growth on large datasets. Model nodes release their training arrays after fitting because the fitted estimator is sufficient for inference. The backend also separates output-container allocation from input copying. For preprocessing, the helper `shouldCopyPreprocessInput` decides whether a defensive copy is needed. Scikit-learn transformers normally return transformed data without mutating the original input, so unnecessary copies can be skipped for those objects. Custom functions remain protected when mutation is possible.

3.1.5 Saving and Loading Pipelines

Pipeline objects can be saved as pickle files. Native graph pickling is provided through the PyBind binding for `graph.Graph`, where C++ exposes a `py::pickle` state pair. During `pickle.dump`, Python asks the bound object for its registered pickle state, and the C++ graph returns a Python dictionary that describes the graph structure and stored objects. During load, the inverse state function rebuilds the graph object.

Custom preprocessing functions are handled by storing source code when normal pickle cannot serialize the function object directly. The save path extracts a top-level function or simple lambda source, and the load path executes that source inside a controlled namespace to recover the callable. Functions with closures are rejected because their external captured variables would not be available from source text alone.

3.1.6 How to Use Accelera Pipe

Algorithm 3: Accelera Pipe example

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.svm import SVC
from accelera.src.accelera_pipe.core.pipeline import Pipeline

accpipe = Pipeline()
results, executed = (
    accpipe
    .branch(
        "preprocessing",
        accpipe.preprocess("standard", StandardScaler(), branch=True),
        accpipe.preprocess("minmax", MinMaxScaler(), branch=True),
    )
    .model("svc", SVC())
    .predict("predict", test_data=X_val)
    .metric("accuracy", "accuracy_score", y_true=y_val)
)(X_train, y_train, select_strategy="max")

test_results = executed(X_test, y_true=y_test)
executed.save("pipeline.pkl")
loaded = executed.load("pipeline.pkl")
```

3.2 Code Parallelizer Module

3.2.1 Module Responsibility

The Code Parallelizer transforms supported loop-heavy source code into OpenMP-annotated C++. The main Python orchestration file is `accelera/src/utils/parallelizer.py`. Python input is first lowered by `accelera/src/utils/py2cpp_converter.py`. Native loop extraction and pragma insertion are implemented in C++ under `src/ast/` and `src/utils/code_parallelizer_utils.cpp`. The module can process files or in-memory code strings.

The workflow has four stages:

1. Convert supported Python code to C++ when the input is Python.
2. Use Clang-based analysis to extract loops and loop metadata.
3. Predict the loop class using a classifier and rule checks.
4. Insert OpenMP pragmas and return the transformed C++ code.

3.2.2 Loop Analysis and Pragma Selection

Loop extraction records syntactic and semantic features such as loop nesting, array access, assignments, reduction patterns, loop-carried dependencies, function calls, and scalar updates. These features are passed to the classifier service. The classifier predicts one of three classes: `none`, `parallel_for`, or `reduction`. The rule layer then validates the predicted class before emitting the pragma.

Algorithm 4: Loop classification and OpenMP annotation

```
1: Input: source program P
2: Output: OpenMP-annotated C++ program P'
3:
4: 1: if P is Python then
5: 2:   convert the supported Python subset to C++
6: 3: end if
7: 4: Parse C++ source using the Clang frontend action
8: 5: for each extracted loop L do
9: 6:   compute structural and dependency features F(L)
10: 7:   request class c from the OpenMP classifier
11: 8:   if c is parallel_for and dependency checks pass then
12: 9:     insert "#pragma omp parallel for"
13: 10:   else if c is reduction and reduction variable is valid then
14: 11:     insert "#pragma omp parallel for reduction(...)"
15: 12:   else
16: 13:     keep the loop sequential
17: 14:   end if
18: 15: end for
19: 16: Return the rewritten C++ source
```

3.2.3 Classifier Model and Rule-Based Decision

The classifier assets are stored under `models/open_mp_classifier/`. The service loads `model.pkl` and `label_encoder.pkl`, exposes a FastAPI prediction endpoint, and maps loop features to one of the supported pragma classes. The trained model is an `XGBClassifier` over a fixed 40-feature loop representation.

An earlier generator trial attempted to generate pragma decisions using a neural sequence model. The model used a custom PyTorch encoder-decoder architecture: a bidirectional LSTM encoder, an attention-based LSTM decoder, teacher forcing during training, cross-entropy loss, Adam optimization, gradient clipping, and a custom BPE tokenizer. This approach was rejected for two scientific reasons. First, the available dataset contained about 15,000 samples, which was not enough for a sequence model to generalize reliably to unseen program structures; the model overfit easily. Second, generating extra training programs with AI tools risked injecting generator-specific bias, which would lower generalization on real user code. The final approach therefore uses explicit loop features, a classifier, and rule-based safety checks.

3.2.4 Python-to-C++ Converter Support

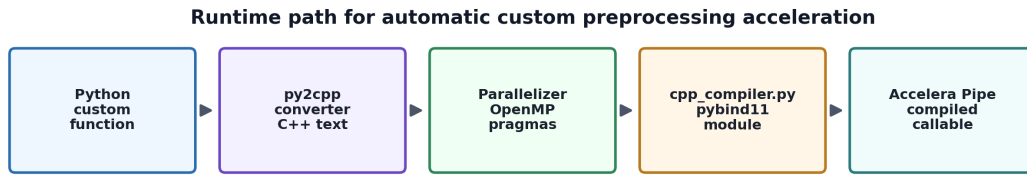
The Python converter intentionally supports a restricted subset. This makes the generated C++ predictable enough for the parallelizer and avoids silently miscompiling dynamic Python behavior.

Table 2: Supported operations in the Python-to-C++ converter

Category	Supported forms	Notes
Values and names	Numeric constants, names, simple lists	Dynamic objects are outside the subset.
Arithmetic	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , <code>**</code>	Power maps to <code>std::pow</code> .
Boolean logic	<code>and</code> , <code>or</code> , <code>not</code>	Emitted as C++ boolean expressions.
Comparisons	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>==</code> , <code>!=</code>	Chained comparisons are restricted.
Calls	Supported built-ins and direct calls	Unsupported calls raise converter errors.
Indexing	Array-style subscripting	Slice semantics are not supported.
Assignment	Simple assignment and selected augmented assignment	Used for loop updates and reductions.
Control flow	<code>if/else</code> , <code>return</code> , <code>for</code> , <code>range(...)</code>	Main loop form used by the parallelizer.
Functions	Plain function definitions	Decorators, closures, and dynamic dispatch are outside the safe subset.

3.2.5 Integration with Accelera Pipe

The parallelizer is integrated into Accelera Pipe for custom preprocessing functions and custom transformer instances. If a preprocessing function is a user-defined Python function, the pipeline attempts to optimize it through the parallelizer. If the object is a custom transformer, the transform method can be optimized and attached back to the instance. This allows the user to write a normal Python preprocessing step while the system compiles the loop-heavy part into a Python-callable C++ extension.



Supported Python loops are converted, parallelized, compiled, cached, and called from the existing pipeline node.

Figure 3: Integration path from Python custom function to compiled Accelera Pipe preprocessing

3.2.6 How to Use the Parallelizer

Algorithm 5: Parallelizer file and in-memory usage

```

from accelera.src.utils.parallelizer import Parallelizer

parallelizer = Parallelizer()

# File mode
cpp_code = parallelizer.parallelize("examples/loop_example.py")

# In-memory code mode
source = """
def normalize_rows(X):
    for i in range(len(X)):
        s = 0
        for j in range(len(X[i])):
            s += X[i][j] * X[i][j]
        norm = s ** 0.5
        for j in range(len(X[i])):
            X[i][j] = X[i][j] / norm
    return X
"""
cpp_code = parallelizer.parallelize(source, file=False)

```

3.3 AutoML Module

3.4 Deployment Module

3.5 Benchmark Platform Module

4 Results and Experiments

4.1 Accelera Pipe Results

The Accelera Pipe experiments compare three implementations under the same train, validation, and test split. The first implementation is a manually written scikit-learn loop over the

candidate pipelines. The second uses `sklearn.pipeline.Pipeline` to represent the same candidates. The third uses Accelera Pipe with graph branches. Each candidate is selected using validation accuracy, and the final selected path is evaluated on the test set. This protocol checks whether the graph execution improves latency without changing the selected model or the measured predictive accuracy.

Table 3: Accelera Pipe latency and memory scaling

Samples	Train	Val	Test	sklearn time	sklearn RSS	Pipeline time	Pipeline RSS	Accelera time	Accelera RSS
1,000	600	200	200	0.06	1.62	0.05	0.16	0.34	19.32
5,000	3,000	1,000	1,000	2.47	3.62	2.12	1.09	2.02	21.62
10,000	6,000	2,000	2,000	2.64	11.25	2.67	2.19	2.10	28.43
50,000	30,000	10,000	10,000	23.96	126.95	23.25	9.75	14.81	209.25
100,000	60,000	20,000	20,000	77.79	139.70	77.10	47.65	67.04	300.93
250,000	150,000	50,000	50,000	803.56	303.95	803.01	28.52	494.64	594.86

Table 3 shows the main performance trend. At very small sizes, Accelera Pipe is slower because the native graph setup, branch creation, and Python/C++ boundary cost dominate the actual model computation. Once the dataset grows, those fixed costs become small compared with fitting multiple candidate branches. At 50,000 samples, Accelera Pipe reduces runtime from 23.96 seconds to 14.81 seconds. At 250,000 samples, it reduces runtime from about 803 seconds to about 495 seconds, saving more than five minutes while selecting the same candidate. The table also shows the current cost of graph-level parallelism: Accelera Pipe uses more RSS memory because multiple branch states and intermediate arrays can exist at the same time. The memory optimizations in the methodology section target exactly this issue.

Table 4: Selected candidate and accuracy across sample sizes

Samples	Selected candidate	Validation accuracy	Test accuracy
1,000	MinMaxScaler → SVC	0.7900	0.8250
5,000	MinMaxScaler → SVC	0.9160	0.9100
10,000	MinMaxScaler → SVC	0.9385	0.9260
50,000	MinMaxScaler → SVC	0.9565	0.9598
100,000	StandardScaler → SVC	0.9675	0.9709
250,000	StandardScaler → SVC	0.9774	0.9768

Table 4 confirms that the speedup does not come from changing the search space. Accelera Pipe evaluates the same candidate pipelines and reports the same validation and test behavior. The selected preprocessing strategy changes from MinMax scaling to standard scaling only when the larger dataset makes StandardScaler with SVC the best validation candidate. This is expected because the candidate set and selection rule are fixed while the data distribution becomes better represented at larger sizes.

Table 5: Largest Accelera Pipe comparison

Implementation	Time (s)	RSS MB	VMS MB	Test accuracy
sklearn manual loop	803.56	303.95	315.41	0.9768
sklearn Pipeline	803.01	28.52	28.61	0.9768
Accelera Pipe	494.64	594.86	787.47	0.9768

Table 5 isolates the largest run: 150,000 training samples, 50,000 validation samples, and 50,000 test samples. Accelera Pipe keeps exactly the same selected path, StandardScaler fol-

lowed by SVC, and the same test accuracy. The runtime drop is 308.91 seconds relative to the manual scikit-learn loop and 308.36 seconds relative to scikit-learn Pipeline. This result is important because it demonstrates the value of graph scheduling on realistic branch-heavy workloads. The memory columns also show the next engineering target: scikit-learn Pipeline is extremely memory efficient, while Accelerate Pipe trades memory for parallel branch execution and native graph state.

4.2 Code Parallelizer Results

The OpenMP classifier was evaluated as a three-class loop classifier. The labels are `none`, `parallel_for`, and `reduction`. The classification report shows that the model is balanced enough to guide the rule layer, while the final pragma decision remains protected by dependency checks.

Table 6: OpenMP classifier report

Class	Precision	Recall	F1-score	Support
none	0.82	0.85	0.84	3048
parallel_for	0.86	0.81	0.83	2696
reduction	0.79	0.82	0.80	775
accuracy			0.83	6519
macro avg	0.82	0.83	0.83	6519
weighted avg	0.83	0.83	0.83	6519

Table 6 should be read as a guidance-quality result, not as a proof that every predicted loop is safe. The classifier is responsible for identifying likely parallelization opportunities. The rule layer is responsible for rejecting unsafe or unsupported cases. This two-stage design is useful because source code parallelization has correctness constraints that pure statistical classification should not decide alone.

Table 7: Hard parallelizer evaluation

File	Baseline ms	Parallel ms	Speedup	Output match
hard_eval_000.cpp	1786.28	195.47	9.138×	true
hard_eval_001.cpp	63.98	31.47	2.033×	true
hard_eval_002.py	42.66	7.67	5.563×	true

Table 7 evaluates harder programs from `data/hard_test_files`. The benchmark compiles and runs both the baseline and the generated parallelized code, then compares their outputs. The first file obtains the largest improvement because its loop body has enough work to amortize OpenMP thread overhead. The second file still improves, but the smaller absolute runtime limits the maximum speedup. The Python-origin file demonstrates the full converter path: Python subset to C++, then loop extraction, then pragma insertion. Across the three files, the evaluation extracted 11 loops, generated 8 pragmas, matched all 8 expected pragmas, reached average similarity 1.0, and obtained an average speedup of 5.578×

Table 8: Parallelizer and Accelera Pipe integration benchmark

Mode	Samples	Time (s)
Python custom preprocessing function	500,000	6.95
Compiled OpenMP kernel only	500,000	0.07
End-to-end compiled run, cold cache	500,000	4.45
End-to-end compiled run, cached module	500,000	2.59

Table 8 separates kernel execution time from end-to-end pipeline time. The 0.07 second measurement is the compiled preprocessing kernel alone and does not include compilation. The 4.45 second run includes conversion, OpenMP parallelization, C++ extension compilation, module loading, and execution. The 2.59 second run benefits from the compiled-module cache, so it represents the more realistic repeated-run cost. This confirms that the integration is useful, but also shows why compilation caching is necessary: without caching, compile latency hides part of the kernel speedup.

4.3 AutoML Results

4.4 Deployment Results

4.5 Benchmark Platform Results

5 Conclusion

This thesis organized Accelera around its five project modules and documented the current methodology and results for Accelera Pipe and the Code Parallelizer. Accelera Pipe provides a graph-based execution model for branch-heavy machine learning pipelines, supports fitted executed graphs, and includes memory optimizations such as consumer-count based release, duplicate first-node sharing, optional cache behavior, and reduced defensive copying. The Code Parallelizer provides a source-to-source path from supported Python or C++ loops to OpenMP-annotated C++, using converter rules, loop feature extraction, classifier guidance, and safety checks.

The experiments show that Accelera Pipe can substantially reduce runtime on large branch-selection workloads while preserving the selected model and test accuracy. They also show that memory remains the most important optimization direction for the graph backend. The parallelizer experiments show correct output preservation on hard evaluation files, meaningful speedups, and a practical integration path where custom Accelera Pipe preprocessing code can be compiled into cached OpenMP-backed Python modules.